

# Stack

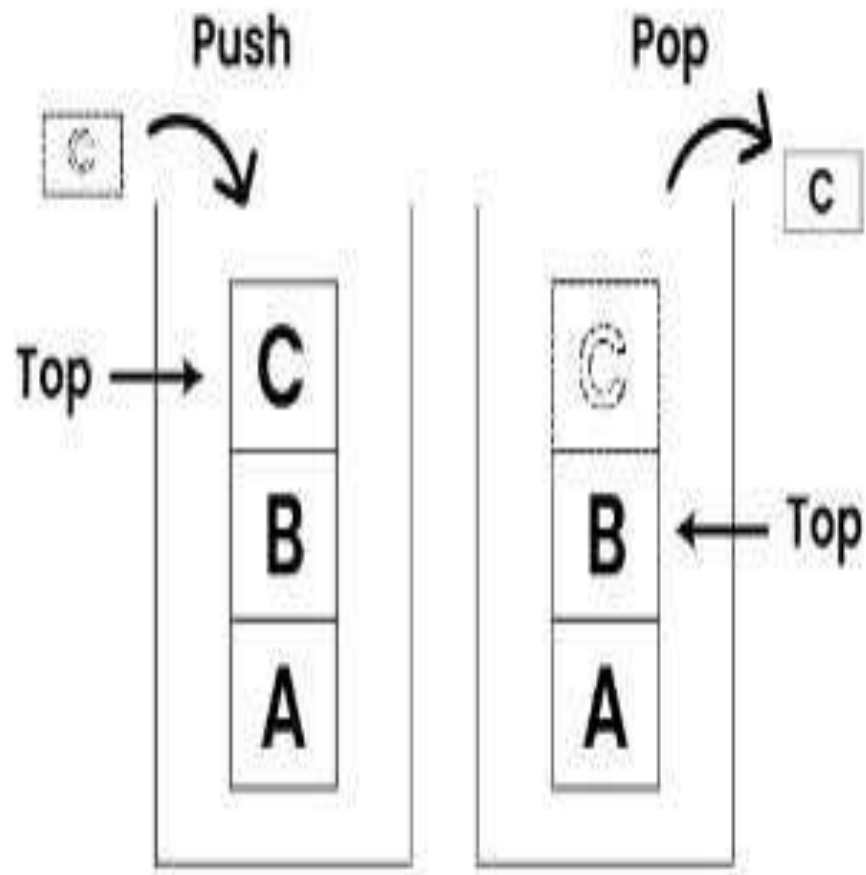


“A stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle.”

It behaves like a stack of plates, where the last plate added is the first one to be removed.

# What is Stack?

A Complete Tutorial



# Advantages of Stack:



- **Simplicity:** Stacks are a simple and easy-to-understand data structure, making them suitable for a wide range of applications.
- **Efficiency:** Push and pop operations on a stack can be performed in constant time ( $O(1)$ ), providing efficient access to data.
- **Last-in, First-out (LIFO):** Stacks follow the LIFO principle, ensuring that the last element added to the stack is the first one removed. This behavior is useful in many scenarios, such as function calls and expression evaluation.
- **Limited memory usage:** Stacks only need to store the elements that have been pushed onto them, making them memory-efficient compared to other data structures.

# Disadvantages of Stack:

- **Limited access:** Elements in a stack can only be accessed from the top, making it difficult to retrieve or modify elements in the middle of the stack.
- **Potential for overflow:** If more elements are pushed onto a stack than it can hold, an overflow error will occur, resulting in a loss of data.
- **Not suitable for random access:** Stacks do not allow for random access to elements, making them unsuitable for applications where elements need to be accessed in a specific order.
- **Limited capacity:** Stacks have a fixed capacity, which can be a limitation if the number of elements that need to be stored is unknown or highly variable.



# Types of Stack

- **Fixed Size Stack** : As the name suggests, a fixed size stack has a fixed size and cannot grow or shrink dynamically. If the stack is full and an attempt is made to add an element to it, an overflow error occurs. If the stack is empty and an attempt is made to remove an element from it, an underflow error occurs.



- 
- **Dynamic Size Stack** : A dynamic size stack can grow or shrink dynamically. When the stack is full, it automatically increases its size to accommodate the new element, and when the stack is empty, it decreases its size. This type of stack is implemented using a linked list, as it allows for easy resizing of the stack.

# Key Operations on Stack Data Structures



- **push()**: When we insert an element in a stack then the operation is known as a push. If the stack is full then the overflow condition **occurs**.
- **pop()**: When we delete an element from the stack, the operation is known as a pop. If the stack is empty means that no element exists in the stack, this state is known as an underflow state.
- **isEmpty()**: It determines whether the stack is empty or not.



- **isFull()**: It determines whether the stack is full or not.
- **peek()**: It returns the element at the given position.
- **count()**: It returns the total number of elements available in a stack.
- **change()**: It changes the element at the given position.
- **display()**: It prints all the elements available in the stack.

- **PUSH (S, TOP, X)**

- This procedure inserts an element  $x$  to the top of a stack which is represented by a vector  $S$  containing  $N$  elements with a pointer  $TOP$  denoting the top element in the stack.

1. [Check for stack overflow]

If  $TOP \geq N$

Then write ('STACK OVERFLOW')

Return

2. [Increment TOP]

$TOP \leftarrow TOP + 1$

3. [Insert Element]

$S[TOP] \leftarrow X$

4. [Finished]

Return



- **POP (S, TOP)**

- This function removes the top element from a stack which is represented by a vector S and returns this element. TOP is a pointer to the top element of the stack.

1. [Check for underflow of stack]

    If  $TOP = 0$

    Then Write ('STACK UNDERFLOW ON POP')

        Take action in response to underflow

        Return

2. [Decrement Pointer]

$TOP \leftarrow TOP - 1$

3. [Return former top element of stack]

    Return (S[TOP + 1])





- **PEEK (S, TOP, I)**
- This function returns the value of the  $i$ th element from the TOP of the stack which is represented by a vector S containing N elements. The element is not deleted by this function.

1. [Check for stack Underflow]

If  $TOP - I + 1 \leq 0$

Then Write ('STACK UNDERFLOW ON PEEP')

Take action in response to Underflow

Exit

2. [Return  $I^{\text{th}}$  element from top of the stack]

Return (S[TOP - I + 1])

# Application of Stack

- **Function calls:** Stacks are used to keep track of the return addresses of function calls, allowing the program to return to the correct location after a function has finished executing.
- **Recursion:** Stacks are used to store the local variables and return addresses of recursive function calls, allowing the program to keep track of the current state of the recursion.
- **Expression evaluation:** Stacks are used to evaluate expressions in postfix notation (Reverse Polish Notation).



- **Syntax parsing:** Stacks are used to check the validity of syntax in programming languages and other formal languages.
- **Memory management:** Stacks are used to allocate and manage memory in some operating systems and programming languages.



# Polish Notation

- Polish Notation is a mathematical notation for expressing arithmetic, logical, and algebraic equations.
- It was introduced by the Polish mathematician **Jan Łukasiewicz** in the early **20th century**.
- This notation is particularly useful in computer science for parsing and evaluating mathematical expressions because it eliminates the need for parentheses and operator precedence rules.



Here are the main types of Polish Notation used in data structures



### **1)Prefix Notation (Polish Notation)**

- Operators appear before their operands.
- Example:  $+ A B$

### **2)Infix Notation**

- Operators appear between their operands (standard arithmetic notation).
- Example:  $A + B$

### **3)Postfix Notation (Reverse Polish Notation)**

- Operators appear after their operands.
- Example:  $A B +$

# Prefix Notation (Polish Notation)



- In Prefix Notation, the operator is placed before its operands.
- Parentheses are not required because the position of the operator defines the evaluation order.
- Format:
- `<operator> <operand1> <operand2>`
- Example: `+ A B`
- Evaluation Rules:
  - Start reading the expression from right to left.
  - When encountering an operator, ensure it has the required number of operands to its right.
  - Evaluate the operator using the operands and replace the expression with the result.

# Postfix Notation (Reverse Polish Expression)



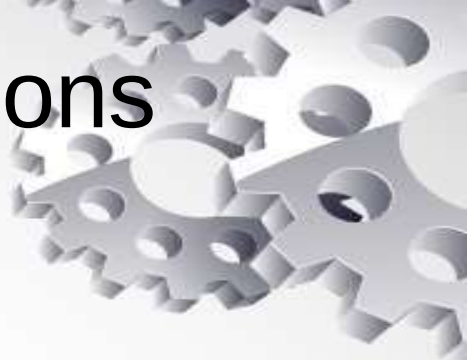
- In Postfix Notation, the operator is placed after its operands.
- Parentheses are not required because the position of the operator defines the evaluation order.
- Format:
  - `<operand1> <operand2> <operator>`
- Example: `A B +`
- Evaluation Rules:
  - Start reading the expression from left to right.
  - Push operands onto a stack.
  - When encountering an operator, pop the required number of operands from the stack.
  - Apply the operator and push the result back onto the stack.
  - The final result is the only value remaining on the stack.

# Conversion of infix to postfix notation

- we will be converting the infix expression to its equivalent postfix expression.
- The precedence order is as follows: (^) has the highest precedence and is evaluated from right to left, (\* and /) come next with left to right associativity, and (+ and -) have the lowest precedence with left to right associativity.



# Conversion of Infix to Prefix and Postfix Notations



1. Infix Expression:  $A+B\times C$

Step 1 (Postfix):

- First, handle multiplication  $B\times C$ , which becomes  $B\ C\ ^*$ .
- Then add  $A+(B\times C)$ , which becomes  $A\ B\ C\ ^*\ +$ .

**Postfix:**

**$A\ B\ C\ ^*\ +$**

Step 2 (Prefix):

- First, handle multiplication, which becomes  $^*\ B\ C$ .
- Then add  $A+(B\times C)$ , which becomes  $+ \ A\ ^*\ B\ C$ .

**Prefix:**

**$+ \ A\ ^*\ B\ C$**

**Input:**  $s = "a*(b+c)/d"$

**Output:**  $abc+*d/$

**Explanation:** The expression is  $a * (b + c) / d$ . First, inside the brackets,  $b + c$  becomes  $bc+$ . Now the expression looks like  $a * (bc+) / d$ . Next, multiply  $a$  with  $(bc+)$ , so it becomes  $abc+*$ . Finally, divide this result by  $d$ , so it becomes  $abc+*d/$ .

**Input:**  $s = "a+b*c+d"$

**Output:**  $abc^*+d+$

**Explanation:** The expression  $a + b * c + d$  is converted by first doing  $b * c \rightarrow bc^*$ , then adding  $a \rightarrow abc^*+$ , and finally adding  $d \rightarrow abc^*+d+$ .



# Comparison of Polish and Reverse Polish Expressions



| Notation                 | Expression (A + B * C) | Evaluation Order                         |
|--------------------------|------------------------|------------------------------------------|
| Prefix (Polish)          | + A * B C              | Operators are processed before operands. |
| Postfix (Reverse Polish) | A B C * +              | Operators are processed after operands.  |

# Applications of Polish and Reverse Polish Notation



- **Expression Evaluation:**

- Used in calculators and compilers for efficient arithmetic computation.
- Polish and Reverse Polish expressions ensure unambiguous parsing, avoiding errors due to operator precedence.

- **Intermediate Code Representation:**

- Polish and Reverse Polish notations are used as intermediate forms in compilers.
- Simplifies the process of optimization and code generation in compiler design.

- **Stack-Based Algorithms:**

- Reverse Polish Notation is particularly useful for stack-based evaluations.
- Facilitates efficient execution of expressions in stack machines like JVM (Java Virtual Machine).



## ■ **Programming Languages:**

- functional programming languages utilize Prefix (Polish) Notation.
- Postfix notation is often leveraged in stack-oriented programming languages like Forth.

## ■ **Machine Learning and Symbolic Computation:**

- Utilized for parsing and optimizing mathematical expressions in symbolic computation.
- Popular in AI and symbolic algebra systems like Mathematica and MATLAB.

## ■ **Hardware Design:**

- Employed in the design of arithmetic circuits and calculators.
- Helps simplify hardware implementation of expression evaluation.

## ■ **Embedded Systems:**

- Used for compact representation and evaluation of mathematical expressions where memory is limited.